A Contention-Aware Duplication Heuristic for DAG Scheduling on NoC Systems

Barış Büyükyılmaz* and Oğulcan Uğuroğlu†
Department of Computer Engineering, Hacettepe University, Ankara, Türkiye
E-mails: *barisbuyukyilmaz@gmail.com, †ogulcan.uguroglu7@gmail.com

***Abstract*—Task scheduling of Directed Acyclic Graphs (DAGs) on multiprocessor systems is a well-known optimization problem in parallel and embedded computing. Task duplication is a commonly used technique to reduce inter-processor communication by replicating selected tasks across multiple processors [1]–[3].**

However, most classical duplication-based approaches rely on simplified communication models and ignore network contention. In modern Network-on-Chip (NoC) based systems, communication delays are strongly affected by shared network resources, routing conflicts, and bandwidth limitations [4]. As a result, duplication decisions that are beneficial under idealized models may lead to increased congestion and degraded performance in realistic systems.

In this work, we study contention-aware task duplication for DAG scheduling on NoC architectures. Building on contention-aware scheduling models, we consider both computation and communication effects during scheduling and investigate how duplication decisions can be improved under network contention. The proposed approach uses a list-based scheduling framework together with a contention-aware duplication heuristic that evaluates the trade-off between execution-time improvement and communication-induced congestion.

The goal of this project is to reduce makespan in communication-intensive scenarios while avoiding unnecessary duplication. The approach is evaluated on synthetic DAG workloads under different communication-to-computation ratio (CCR) settings and NoC configurations.

***Index Terms*—task scheduling, task duplication, Network-on-Chip, contention-aware scheduling, DAG scheduling, NoC, MPSoC.**

I. Introduction

Task scheduling of Directed Acyclic Graphs (DAGs) on multiprocessor systems is a fundamental problem in parallel and embedded computing. In this problem, tasks are represented as nodes and dependencies as edges, and the objective is typically to minimize the overall execution time (makespan) while satisfying precedence and resource constraints.

Task duplication is a well-established technique used to reduce communication overhead in such systems. By duplicating selected parent tasks across processors, communication can be avoided or reduced, allowing successor tasks to start earlier. Classical duplication-based scheduling algorithms such as DBUS [1], HCPFD [2], and HLD [3] have demonstrated that duplication can significantly improve scheduling performance, particularly in communication-intensive workloads.

Despite their effectiveness, these approaches rely on simplified communication models that assume constant communication cost or contention-free networks. Such assumptions are not realistic for modern multiprocessor systems, especially Network-on-Chip (NoC) based architectures, where communication resources are shared and contention plays a critical role in determining performance.

Contention-aware scheduling has been proposed to address this limitation by explicitly modeling communication conflicts and network behavior. In particular, Sinnen et al. [4] introduced a contention-aware scheduling model that incorporates communication resource constraints into the scheduling process by scheduling communication edges on network links. This work shows that considering contention leads to more accurate and efficient schedules.

Under contention-aware models, task duplication becomes even more important, since reducing inter-processor communication can also reduce network contention. However, duplication decisions must be made carefully. Excessive or poorly placed duplication may increase resource usage and may not lead to performance improvements.

Motivated by this observation, this project investigates a contention-aware task duplication heuristic for DAG scheduling on NoC systems. The key idea is that duplication decisions should not be based solely on execution-time improvement, but should also consider their impact on network contention.

The proposed approach builds on a list-based scheduling framework similar to HEFT-like methods [5], and augments it with a contention-aware duplication heuristic. For each duplication candidate, the method evaluates both the reduction in earliest finish time and the associated communication cost under contention.

Unlike optimization-based approaches such as [6], which formulate duplication scheduling as an Integer Linear Programming (ILP) problem, this work focuses on a heuristic-based approach that is simpler to implement and suitable for practical scheduling scenarios.

The main objective of this project is to develop a consistent and implementable contention-aware duplication strategy and to evaluate its effectiveness under different workload and network conditions.

The main contributions of this work are as follows:

- A contention-aware task duplication heuristic for DAG scheduling on NoC-based multiprocessor systems.
- A scheduling formulation that jointly considers computation, communication, and duplication under network contention.
- An implementation-oriented evaluation framework based on synthetic DAG workloads with varying communica-

tion characteristics.

II. Related Work

Task scheduling for Directed Acyclic Graphs (DAGs) has been extensively studied in multiprocessor and heterogeneous computing systems. Existing approaches can be categorized into duplication-based scheduling, resource-aware duplication strategies, and contention-aware scheduling methods.

A. Duplication-Based Scheduling

Duplication-based scheduling algorithms aim to reduce communication overhead by replicating tasks across multiple processors. Early works such as the *Duplication-Based Bottom-Up Scheduling* (DBUS) algorithm [1], the *Heterogeneous Critical Parent with Fast Duplication* (HCPFD) algorithm [2], and the *Heterogeneous Limited Duplication* (HLD) approach [3] exploit duplication to place parent tasks closer to dependent tasks, thereby reducing communication delays and improving makespan.

Subsequent works further refined duplication strategies by introducing selective duplication mechanisms. For example, improved duplication strategies reduce redundant task copies while maintaining performance [7]. Similarly, the *Heterogeneous Earliest Finish with Duplication* (HEFD) algorithm [5] extends list scheduling by incorporating duplication into heterogeneous environments.

Despite their effectiveness, these approaches rely on simplified communication models that assume constant communication costs and do not account for network contention.

B. Resource- and Energy-Aware Duplication

More recent approaches address the inefficiencies of aggressive duplication. Resource-aware scheduling methods aim to reduce redundant task replication while preserving scheduling performance [8]. Energy-aware duplication algorithms further consider the trade-off between performance and energy consumption [9].

These methods demonstrate that duplication must be carefully controlled, as excessive replication can degrade performance. However, they still do not consider network-level contention effects.

C. Contention-Aware Scheduling

Contention-aware scheduling approaches explicitly model communication conflicts and network behavior. Sinnen et al. [4] propose a contention-aware duplication scheduling algorithm that integrates communication contention into scheduling decisions. This work highlights the importance of modeling realistic communication behavior.

However, duplication decisions in such approaches are still primarily driven by scheduling heuristics and are not explicitly controlled based on network congestion.

D. NoC-Aware Scheduling Optimization

Tang et al. [6] extend scheduling models to Network-on-Chip (NoC) architectures and propose an optimization framework for duplication-based schedules. Their work considers both contention-free and contention-aware scenarios using Integer Linear Programming (ILP).

While this approach provides accurate modeling of NoC behavior, it assumes that duplication strategies are given and focuses on optimizing scheduling rather than improving duplication decisions.

E. Position of This Work

Although significant progress has been made in both duplication-based and contention-aware scheduling, the interaction between task duplication and network contention remains insufficiently explored. Existing methods either perform duplication without considering contention or consider contention without explicitly controlling duplication decisions.

This work addresses this gap by integrating network congestion directly into duplication decisions. Unlike prior approaches, duplication is explicitly regulated through a contention-aware decision rule, enabling more selective and efficient task replication.

A comparative summary of the discussed scheduling approaches is provided in Table I, highlighting key differences in duplication handling, contention awareness, and NoC modeling.

TABLE I
Comparison of scheduling approaches in terms of duplication, contention awareness, and NoC modeling capabilities

Method	Duplication	Contention-Aware	NoC Model	Resource-Aware
HEFT	✗	✗	✗	✗
DBUS / HCPFD / HLD	✓	✗	✗	✗
Bansal (Selective Dup.)	✓	✗	✗	✗
Mei et al.	✓	✗	✗	✓
Liang et al.	✓	✗	✗	✓
Sinnen et al.	✓	✓	✗	✗
Tang et al.	✗	✓	✓	✗
This Work	✓	✓	✓	✓

As shown in Table I, existing approaches do not simultaneously address duplication decision and network contention, which motivates the proposed method.

III. Problem Definition

This section formally defines the DAG scheduling problem under a contention-aware communication model with task duplication.

A. System Model

The application is represented as a Directed Acyclic Graph (DAG):

$$G = (V, E, w, c)$$

where:

- $V = \{v_1, v_2, \ldots, v_n\}$: set of tasks
- $E \subseteq V \times V$: precedence constraints

- w_i: computation cost of task v_i
- c_{ij}: communication volume from v_i to v_j

The target platform consists of processors:

$$P = \{p_1, p_2, \ldots, p_m\}$$

connected via a Network-on-Chip (NoC). Communication between processors is subject to routing delays and contention.

Tasks may be duplicated on multiple processors to reduce communication overhead.

B. Decision Variables

- $x_{ik} \in \{0, 1\}$: task v_i has an instance on processor p_k
- $y_{ik} \in \{0, 1\}$: processor p_k executes the primary instance of task v_i
- s_i^k: start time of task v_i on processor p_k, if an instance exists
- f_i^k: finish time of task v_i on processor p_k, if an instance exists

The processor assigned to the primary execution of task v_i is denoted by $proc(i)$, where $y_{ik} = 1$.

C. Objective

The objective is to minimize the makespan:

$$F = \max_{v_i \in V} f_i^{proc(i)}$$

D. Constraints

1) Primary Assignment Constraint: Each task has exactly one primary execution:

$$\sum_{k=1}^{m} y_{ik} = 1 \qquad \forall v_i \in V$$

2) Instance Existence Constraint: A primary assignment implies the existence of an instance:

$$y_{ik} \leq x_{ik} \qquad \forall v_i \in V,\ p_k \in P$$

3) Timing Constraint: For each existing instance:

$$f_i^k = s_i^k + w_i$$

4) Processor Constraint: Tasks assigned to the same processor cannot overlap in execution.

5) Precedence Constraint (Duplication-Aware): A task can start only after receiving data from all its parent tasks. For each parent, the earliest available data from any duplicated instance is considered:

$$s_j \geq \max_{v_i \in pred(v_j)} \left(\min_{k:\, x_{ik}=1} \left(f_i^k + D_{ij}^{eff}(k, proc(j)) \right) \right)$$

E. Communication Model

Communication delay between processors p_k and p_l is defined as:

$$D_{ij}^{eff}(k, l) = D_{ij}^{base}(k, l) + CP_{ij}(k, l) \tag{1}$$

The base communication delay is modeled as:

$$D_{ij}^{base}(k, l) = \alpha \cdot hops_{kl} + \beta \cdot c_{ij} \tag{2}$$

The proposed communication model is inspired by contention-aware scheduling frameworks in which communication edges are explicitly scheduled on network links and the resulting edge finish time determines data availability at successor tasks. In such models, communication delay emerges from link-level contention and routing decisions.

In this work, instead of performing full link-level edge scheduling during each scheduling step, we adopt a lightweight approximation of contention effects. Communication delay is estimated using route-level congestion indicators associated with the routing path r_{kl} between processors p_k and p_l.

The contention penalty is defined as:

$$CP_{ij}(k, l) = \lambda_1 U(r_{kl}) + \lambda_2 B(r_{kl}) + \lambda_3 L(r_{kl}) \tag{3}$$

where:

- $U(r_{kl})$ denotes the average link utilization along the routing path, representing the fraction of occupied bandwidth on the traversed links.
- $B(r_{kl})$ represents the average buffer occupancy along the path and serves as a proxy for queueing delays at intermediate routers.
- $L(r_{kl})$ denotes the cumulative traffic load on the routing path, measured as the intensity of concurrent communications sharing the same links.

All congestion indicators are estimated based on the current partial schedule during tentative task placement and duplication evaluation. Previously scheduled communications contribute to link utilization, buffer occupancy, and traffic load.

The coefficients λ_1, λ_2, and λ_3 control the relative importance of each contention component and allow balancing different sources of network congestion.

This formulation approximates the behavior of contention-aware communication models while maintaining computational efficiency, making it suitable for heuristic-based scheduling.

F. Duplication Decision Rule

Duplication decisions are based on the improvement in earliest finish time (EFT):

$$\Delta EFT = EFT^{no\text{-}dup} - EFT^{dup}$$

Duplication is applied if:

$$\Delta EFT > 0$$

G. Consistency Note

Duplication decisions are evaluated using tentative scheduling, where communication delays and contention effects are estimated before committing to duplication.

The effect of duplication under a contention-aware communication model is illustrated in Fig. 1. Duplicated instances of a task allow different communications to originate from different processors, which can reduce communication delays for successor tasks.

IV. System Architecture and Design

This section presents the overall system architecture and design of the proposed contention-aware duplication scheduling framework. The system is modeled as a scheduling pipeline operating on a DAG application and a Network-on-Chip (NoC) based multiprocessor platform.

A. Hardware/Software Architecture

The overall system operates on a multiprocessor platform connected through a Network-on-Chip (NoC).

The hardware architecture consists of:

- Multiple processing elements $P = \{p_1, p_2, \ldots, p_m\}$
- A 2D mesh NoC topology
- Routers and communication links supporting deterministic routing

On the software side, the system processes a DAG-based application, where tasks represent computation units and edges represent data dependencies.

The system includes the following components:

- **DAG Analyzer:** extracts dependencies and computes task priorities
- **Base Scheduler:** generates an initial schedule using a HEFT-like algorithm
- **Duplication Decision Module:** evaluates candidate duplications by estimating their impact on earliest finish time (EFT)
- **NoC Communication Model:** estimates communication delays and contention effects

The key contribution of this work lies in the duplication decision module, which uses a contention-aware communication model to guide duplication decisions indirectly through EFT evaluation.

B. Dataflow and System Model

The system follows a pipeline-based dataflow model, where scheduling decisions are performed in sequential stages.

The dataflow can be described as:

1) The input DAG is analyzed to extract task and communication parameters
2) The base scheduler generates an initial mapping using earliest finish time (EFT)
3) Candidate duplications are evaluated for each task
4) Duplication decisions are made based on the improvement in EFT
5) The final schedule is produced

Communication is explicitly modeled, and contention effects are incorporated through the NoC communication model.

C. Timing and Execution Model

The system follows a static scheduling model, where all scheduling decisions are made offline.

The execution model assumes:

- Non-preemptive task execution
- Deterministic computation times
- Communication delays dependent on routing and contention

A task can only begin execution after all its parent tasks have completed and the required data has been received.

D. Model of Computation

The system follows a dataflow model of computation:

- Nodes represent computational tasks
- Edges represent data dependencies
- Task execution is triggered by data availability

This model enables explicit handling of precedence constraints and communication effects.

E. Assumptions and Design Limitations

The proposed system is based on the following assumptions:

- The task graph is static and known in advance
- Execution times are deterministic
- Network topology is fixed (2D mesh)
- Routing is deterministic

The main limitations are:

- Dynamic or runtime scheduling is not considered
- Contention is estimated rather than measured at runtime
- Adaptive or iterative optimization is not included

F. Resource Model

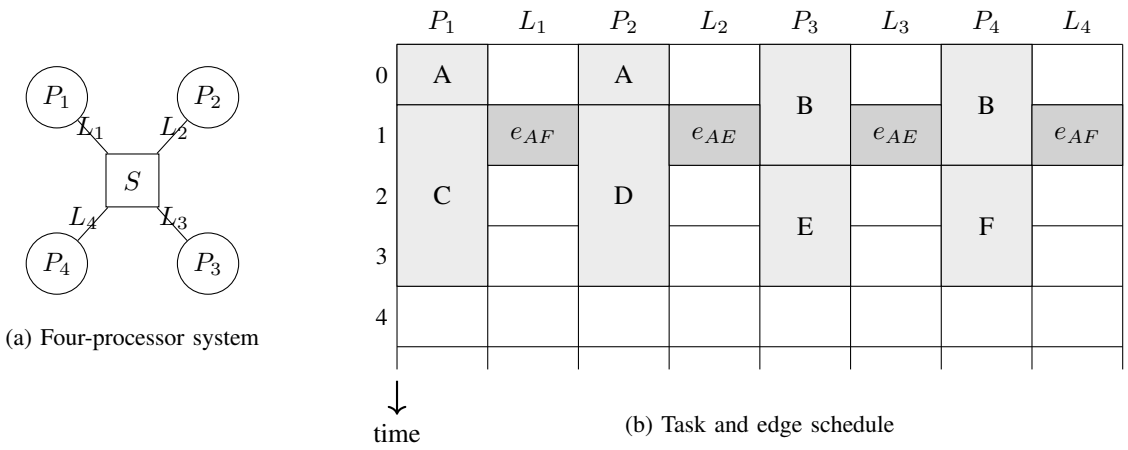
The system assumes limited computational and communication resources:

- **Processing resources:** Each processor executes one task at a time
- **Communication bandwidth:** NoC links are shared and subject to contention
- **Memory usage:** Task duplication increases memory consumption

The proposed approach accounts for communication resource usage through the contention-aware communication model, which influences duplication decisions via EFT evaluation.

V. Methodology

This work improves task duplication decisions in DAG scheduling by incorporating contention-aware communication effects into earliest finish time (EFT) evaluation. A standard list-based scheduler is used as the baseline, and duplication decisions are made based on their impact on EFT.



(a) Four-processor system

(b) Task and edge schedule

Fig. 1. Illustration of task duplication under the contention-aware model. Duplicated instances of task A allow different communications to be sent from different processors, reducing communication delay for successor tasks.

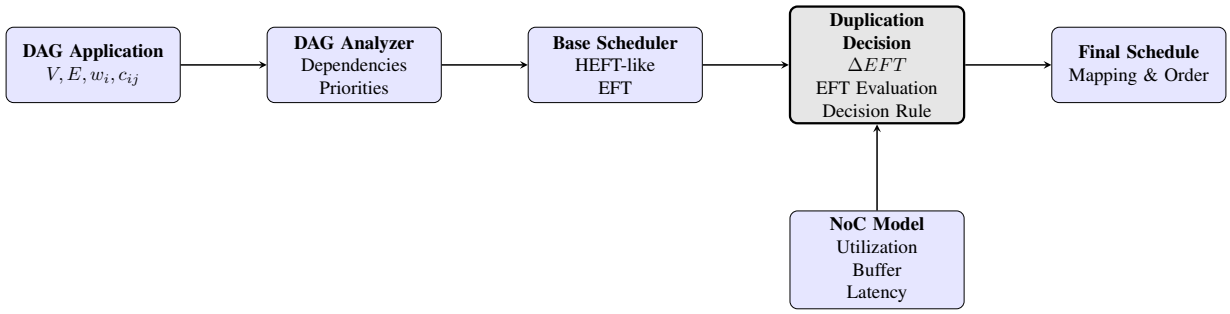


Fig. 2. System architecture of the proposed contention-aware duplication framework. A DAG application is analyzed and scheduled using a base HEFT-like scheduler. Candidate duplications are evaluated based on their impact on earliest finish time (EFT), where communication delays are estimated using a contention-aware NoC model.

A. Optimization Pipeline

The proposed scheduling pipeline consists of four stages:

1) **Input:** A DAG $G = (V, E)$ with computation costs w_i and communication volumes c_{ij}, along with the NoC topology and routing model.
2) **Base Scheduling:** Tasks are ordered based on priority (e.g., upward rank as in HEFT) and scheduled using a list-based approach that minimizes earliest finish time.
3) **Duplication Evaluation:** For each task v_j, duplication of its parent tasks v_i is evaluated using a contention-aware EFT model.
4) **Output:** A final schedule including task-to-processor mapping, execution order, and selected duplicated tasks.

B. Duplication Evaluation Strategy

For each task v_j and each parent task v_i, duplication candidates are evaluated on different processors.

The evaluation is performed using *tentative scheduling*, where the effect of duplication is estimated before committing to the final schedule.

The importance of selecting the appropriate source instance for communication is illustrated in Fig. 3. When multiple communications compete for the same network link, contention may delay data transfers and increase the overall schedule length. Therefore, duplication decisions must consider not only execution time but also communication contention effects through EFT estimation.

For each processor p_k, the following steps are performed:

- Compute $EFT_j^{no\text{-}dup}$: earliest finish time of v_j without duplicating v_i
- Compute $EFT_{j,k}^{dup}$: earliest finish time of v_j assuming v_i is duplicated on p_k
- Compute the improvement:

$$\Delta EFT_{ijk} = EFT_j^{no\text{-}dup} - EFT_{j,k}^{dup}$$

Duplication is applied if:

$$\Delta EFT_{ijk} > 0$$

This ensures that duplication is only performed when it leads to an actual reduction in completion time under the contention-aware communication model.

C. Algorithm Description

1: Compute task priorities (e.g., upward rank)
2: Initialize empty schedule
3: **for** each task v_j in priority order **do**

for each parent task v_i of v_j **do**
  for each processor p_k **do**
    Tentatively evaluate schedule without duplication
    Compute $EFT_j^{no\text{-}dup}$
    Tentatively duplicate v_i on p_k
    Estimate communication delays using the contention-aware model
    Compute $EFT_{j,k}^{dup}$
    Compute:

$$\Delta EFT_{ijk} = EFT_j^{no\text{-}dup} - EFT_{j,k}^{dup}$$

    if $\Delta EFT_{ijk} > 0$ **then**
      Mark duplication candidate
    end if
  end for
end for
Select best processor for v_j based on minimum EFT
Apply selected duplications
Commit task v_j to the schedule
end for
Return final schedule

D. Discussion

The proposed method differs from classical duplication-based scheduling approaches in that duplication decisions are explicitly evaluated using a contention-aware communication model.

Rather than applying duplication heuristically or aggressively, the method selectively performs duplication only when it leads to a measurable improvement in earliest finish time. This avoids unnecessary task replication and reduces communication congestion in the NoC.

Furthermore, the approach remains compatible with standard list-based scheduling algorithms and can be integrated into existing frameworks with minimal modification.

VI. Experimental Setup

This section describes the evaluation methodology used to assess the effectiveness of the proposed contention-aware duplication strategy.

A. Platform and Tools

The evaluation is conducted using a simulation-based environment.

- **NoC Model:** 2D mesh topology
- **NoC Sizes:** 4×4 and 8×8
- **Routing Algorithm:** Deterministic XY routing
- **Simulation Tool:** NoC simulator (e.g., BookSim or trace-based model)
- **Scheduler:** Custom implementation of HEFT-like scheduling with duplication support

The NoC model is used to estimate communication delays and contention effects, which are incorporated into the earliest finish time (EFT) computation.

B. Workloads

Synthetic DAGs are generated to evaluate scheduling performance under different conditions.

- **Number of tasks:** 20–100 nodes
- **Graph type:** Random DAGs with varying levels of parallelism
- **Computation cost:** Random values within a fixed range
- **Communication volume:** Random values assigned to edges

To evaluate communication sensitivity, different communication-to-computation ratio (CCR) values are used:

- Low CCR (0.1): computation-dominant workloads
- Medium CCR (1.0): balanced workloads
- High CCR (5.0): communication-dominant workloads

C. Evaluation Metrics

The performance is evaluated using the following metrics:

- **Makespan:**

$$F = \max_{v_i \in V} f_i$$

- **Normalized Makespan (Speedup):**

$$Speedup = \frac{F_{baseline}}{F_{proposed}}$$

  where the baseline is the HEFT schedule without duplication.
- **Average Communication Latency:** Average delay of all communication edges under the NoC model.
- **Network Utilization:** Average link utilization across the NoC.
- **Duplication Overhead:** Total number of duplicated task instances:

$$Duplication\ Ratio = \frac{\text{Number of task instances}}{|V|}$$

D. Baselines

The proposed method is compared against the following approaches:

- **HEFT:** List scheduling without duplication [5]
- **Classical Duplication:** Duplication-based scheduling without contention awareness [1], [2]
- **Contention-Aware Scheduling:** Scheduling with contention modeling but without explicit duplication control [4]

E. Evaluation Procedure

For each experiment:

1) Generate a DAG instance
2) Apply baseline scheduling algorithms
3) Apply the proposed duplication-aware scheduling method
4) Estimate communication delays using the NoC model
5) Compute EFT values and construct final schedules
6) Collect performance metrics

Each experiment is repeated over multiple DAG instances, and average results are reported.

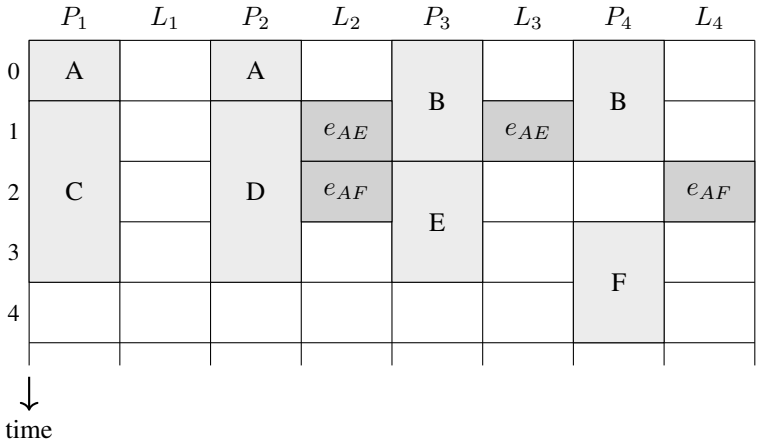


Fig. 3. Contention on one communication link delays e_{AF}, which causes task F to start later and increases the overall schedule length.

F. Expected Results

The proposed method is expected to:

- Reduce makespan in medium and high CCR scenarios
- Achieve higher speedup compared to HEFT and classical duplication
- Reduce unnecessary duplication
- Lower network congestion compared to aggressive duplication strategies

In low-CCR scenarios, limited improvement is expected due to the reduced impact of communication.

VII. System-Level Considerations

This section discusses broader system-level aspects of the proposed approach. While the core contribution focuses on contention-aware duplication decisions, several system-level considerations are relevant for practical deployment.

A. Real-Time Constraints and Latency

The proposed method aims to reduce application makespan, which directly relates to end-to-end latency. By avoiding unnecessary duplication that may increase communication overhead, the method can improve latency in communication-intensive scenarios.

Implemented:

- Makespan optimization through EFT-based duplication decisions

Not implemented / future work:

- Explicit deadline-aware scheduling
- Worst-case execution time (WCET) analysis

B. Reliability and Fault Tolerance

Task duplication can inherently improve reliability by providing redundant execution paths. However, the proposed method uses duplication purely for performance optimization rather than fault tolerance.

Implemented:

- None (duplication is not used for reliability purposes)

Not implemented / future work:

- Fault-tolerant duplication strategies
- Failure-aware scheduling

C. Energy Efficiency

Duplication increases computational workload and may lead to higher energy consumption. By avoiding unnecessary duplication through EFT-based evaluation, the proposed method may indirectly reduce redundant computation, although energy efficiency is not explicitly optimized.

Implemented:

- Indirect reduction of redundant computation

Not implemented / future work:

- Explicit energy-aware optimization
- Power modeling and measurement

D. Security Considerations

The proposed method does not explicitly address security concerns. However, scheduling and communication patterns may influence system-level vulnerabilities.

Implemented:

- None

Not implemented / future work:

- Secure task mapping
- Communication isolation mechanisms

E. Scalability

The proposed method is evaluated on different NoC sizes and remains compatible with standard list-based scheduling frameworks.

Implemented:

- Evaluation on multiple NoC sizes (4×4, 8×8)

Limitations:

- Increased computational cost due to duplication evaluation

F. Other Considerations

The proposed approach is designed for static scheduling scenarios and assumes full knowledge of the task graph and system parameters.

Limitations:

- Not suitable for dynamic or runtime scheduling
- Depends on accuracy of contention estimation

G. Resource Model

The system assumes limited computational and communication resources.

- **Processing resources:** Each processor executes one task at a time
- **Communication bandwidth:** NoC links are shared and subject to contention
- **Memory usage:** Task duplication increases memory consumption

The proposed approach accounts for communication resource usage through the contention-aware communication model, which influences scheduling decisions indirectly via earliest finish time (EFT).

ACKNOWLEDGMENTS

The authors would like to thank the course instructors for their guidance, constructive feedback, and valuable insights throughout the development of this project. Their comments on problem formulation, literature positioning, and system modeling significantly improved the clarity and technical depth of this work.

The authors also acknowledge the use of publicly available research literature and tools that supported the development of the proposed methodology and evaluation framework.

Finally, the authors appreciate the discussions and feedback received during the project process, which helped refine both the technical approach and the presentation of the results.

REFERENCES

[1] D. Bozdag, U. Catalyurek, and F. Ozguner, "A task duplication based bottom-up scheduling algorithm for heterogeneous environments," in *International Conference Proceedings*, 2006.

[2] T. Hagras and J. Janecek, "A high performance, low complexity algorithm for compile-time task scheduling in heterogeneous systems," in *Proceedings of the International Parallel and Distributed Processing Symposium (IPDPS)*, 2004.

[3] S. Bansal, P. Kumar, and K. Singh, "Dealing with heterogeneity through limited duplication for scheduling precedence constrained task graphs," *Journal of Parallel and Distributed Computing*, vol. 65, pp. 479–491, 2005.

[4] O. Sinnen, A. To, and M. Kaur, "Contention-aware scheduling with task duplication," *Journal of Parallel and Distributed Computing*, vol. 71, pp. 77–86, 2011.

[5] X. Tang, K. Li, G. Liao, and R. Li, "List scheduling with duplication for heterogeneous computing systems," *Journal of Parallel and Distributed Computing*, vol. 70, pp. 323–329, 2010.

[6] Q. Tang, S.-F. Wu, J.-W. Shi, and J.-B. Wei, "Optimization of duplication-based schedules on network-on-chip based multi-processor system-on-chips," *IEEE Transactions on Parallel and Distributed Systems*, vol. 28, no. 3, pp. 826–839, 2017.

[7] S. Bansal, P. Kumar, and K. Singh, "An improved duplication strategy for scheduling precedence constrained graphs in multiprocessor systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 14, no. 6, pp. 533–544, 2003.

[8] J. Mei, K. Li, and K. Li, "A resource-aware scheduling algorithm with reduced task duplication on heterogeneous computing systems," *The Journal of Supercomputing*, vol. 68, pp. 1347–1377, 2014.

[9] A. Liang and Y. Pang, "A novel, energy-aware task duplication-based scheduling algorithm of parallel tasks on clusters," *Mathematical and Computational Applications*, vol. 22, no. 1, 2017.